

05506026 Digital Image Processing (Java)

Lab 11

Objectives

1. Understand how to perform accumulative difference image method.

Accumulative Difference Image

Accumulative difference image (ADI) is a technique to segment a sequence of images where the motion takes place. The ADI is formed using the image frames $f(x, y, t_1)$, $f(x, y, t_2)$, ..., $f(x, y, t_n)$ in which the values can be calculated as follow: -

$$A_k(x, y) = \begin{cases} A_{k-1}(x, y) + 1 & \text{if } |R(x, y) - f(x, y, k)| > T \\ A_{k-1}(x, y) & \text{otherwise} \end{cases}$$

$$P_k(x, y) = \begin{cases} P_{k-1}(x, y) + 1 & \text{if } R(x, y) - f(x, y, k) > T \\ P_{k-1}(x, y) & \text{otherwise} \end{cases}$$

$$N_k(x, y) = \begin{cases} N_{k-1}(x, y) + 1 & \text{if } R(x, y) - f(x, y, k) < -T \\ N_{k-1}(x, y) & \text{otherwise} \end{cases}$$

As above, there are three types of ADIs – absolute, positive, and negative. All of them can be implemented using similar approach where the only difference is the condition of the increment intensities. The absolute ADI can be implemented as follow.

ImageManager.java

```
class ImageManager
{
    ...
    public void ADIAbsolute (String[] sequences, int threshold, int step)
    {
        if (img == null) return;

        BufferedImage tempBuf = new BufferedImage(width, height, img.getType());

        for (int n = 0; n < sequences.length; n++)
        {
            BufferedImage otherImage = null;
            try
            {
                otherImage = ImageIO.read(new File(sequences[n]));
            }
        }
    }
}
```

```

        catch (IOException e)
        {
            System.out.println(e);
            return;
        }
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color1 = img.getRGB(x, y);
                int r1 = (color1 >> 16) & 0xff;
                int g1 = (color1 >> 8) & 0xff;
                int b1 = color1 & 0xff;

                int color2 = otherImage.getRGB(x, y);
                int r2 = (color2 >> 16) & 0xff;
                int g2 = (color2 >> 8) & 0xff;
                int b2 = color2 & 0xff;

                int dr = r1 - r2;
                int dg = g1 - g2;
                int db = b1 - b2;

                int dGray = (int) Math.round(0.2126*dr + 0.7152*dg + 0.0722*db);

                if (Math.abs(dGray) > threshold)
                {
                    int currentColor = tempBuf.getRGB(x, y) & 0xff;
                    currentColor += step;

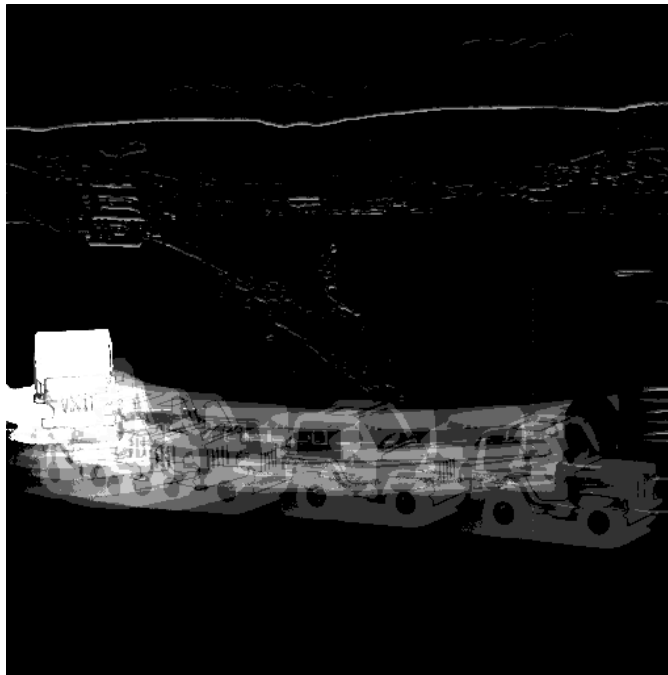
                    currentColor = currentColor > 255 ? 255: currentColor;
                    currentColor = currentColor < 0? 0: currentColor;

                    int newColor = (currentColor << 16) | (currentColor << 8) | cur-
rentColor;
                    tempBuf.setRGB(x, y, newColor);
                }
            }
        }

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                img.setRGB(x, y, tempBuf.getRGB(x, y));
            }
        }
        ...
    }
}

```

The result of using absolute ADI on the sequences of motion.512.bmp with threshold of 25 and step of 50 is as follow.



Quest 015

Implement the method of negative ADI on the sequences of motion.512.bmp with threshold of 25 and step of 50.

The player who finishes this task will be awarded **10 EXP** and **1 Move**.

Deadline: 7 September 2025